CONCEPTO

Python y MySQL

Una variable desaparece cuando termina el programa.
Una base de datos conserva los datos entre ejecuciones.
Una tabla organiza registros en filas y propiedades en columnas.
SQL es el lenguaje para consultar y cambiar esos datos.

CONCEPTO

Una base de datos guarda información

Una variable desaparece cuando termina el programa.
Una base de datos conserva los datos entre ejecuciones.
Una tabla organiza registros en filas y propiedades en columnas.
SQL es el lenguaje para consultar y cambiar esos datos.

CONCEPTO

Claves, relaciones e índices

La clave primaria identifica cada fila sin repetirse.
La clave foránea enlaza un producto con su categoría.
Un índice facilita determinadas búsquedas, con coste de espacio y escritura.
Ejemplo: productos.categoria_id referencia categorias.id.

CONCEPTO

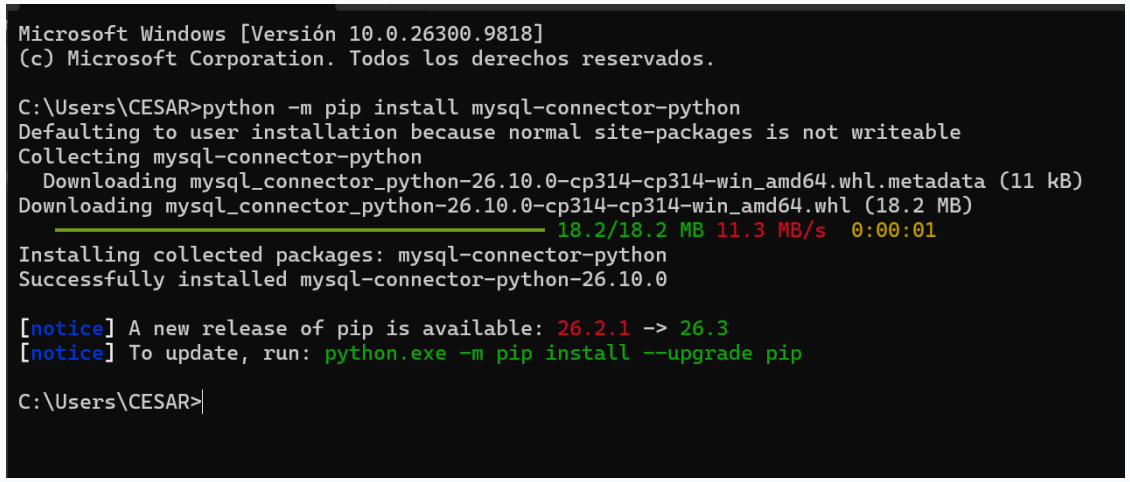
Antes de comenzar: preparar el aula

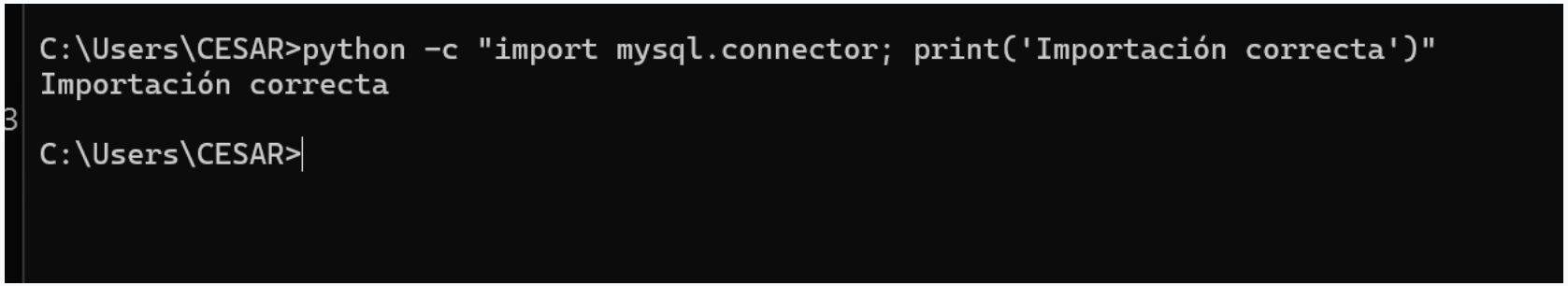
Instalar MySQL Server, Python 3 y opcionalmente Workbench.
Comprobar el servicio MySQL y la conexión local.
Ejecutar 00_preparar.sql como administrador en Workbench.
Crear el usuario con 01_usuario.sql tras elegir su contraseña.
La guía incluye los pasos y soluciones a errores frecuentes.

CONCEPTO

Instalar el conector en la terminal

python -m pip install mysql-connector-python
python -c "import mysql.connector; print('Importación correcta')"





Este comando se escribe en la terminal, fuera del intérprete Python.
Usa el mismo Python con el que ejecutarás los ejercicios.
El paquete se instala como mysql-connector-python.
En el código se importa como mysql.connector.

CONCEPTO

SQL y Python tienen trabajos distintos

DDL define estructuras: CREATE DATABASE y CREATE TABLE.
DML modifica datos: INSERT, UPDATE y DELETE.
SELECT consulta datos.
Python abre la conexión, envía SQL y procesa la respuesta.

Funciones Core del DBA: Lenguajes de Datos

Interacción directa con la estructura y la manipulación de la información.

DDL

Lenguaje de Definición de Datos

Crear y modificar estructuras de la base de datos (tablas, índices, vistas).

DCL

Lenguaje de Control de Datos

Establecer permisos, auditar accesos y garantizar que solo los usuarios autorizados operen.

DML

Lenguaje de Manipulación de Datos

Ejecutar consultas, actualizaciones, inserciones y eliminaciones de registros diarios.

DDL *(data definition language)*

Los comandos **DDL** (*Data Definition Language* o Lenguaje de Definición de Datos) se utilizan para **crear, modificar y destruir la estructura** de los objetos de la base de datos (tablas, vistas, índices, etc.).

A diferencia del DML (que manipula los registros de datos), el DDL define el "contenedor" donde se guardará esa información.

Principales comandos DDL

- **CREATE**: Crea nuevos objetos en la base de datos, como tablas, vistas, índices o bases de datos completas.
- **ALTER**: Modifica la estructura de un objeto ya existente (por ejemplo, añadir una columna a una tabla, cambiar su tipo de datos o renombrarla).
- **DROP**: Elimina por completo un objeto de la base de datos. *(Borrando tanto su estructura como todo su contenido de forma irreversible)*.
- **TRUNCATE**: Vacía todo el contenido de una tabla de forma rápida, pero mantiene intacta la estructura de la columna para futuros registros.

1) CREATE, ALTER, DROP

2) DATABASE, TABLE, VIEW

3) NOMBRE

4) MODIFICADORES....

CREAR TABLA: (SQL)

```
CREATE DATABASE ASGDB;
USE ASGDB;
CREATE TABLE Clientes (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    pais VARCHAR(100) NOT NULL,
    edad INT,
    fecha_contratacion DATE
);
```

CREAR TABLA:

CREATE DATABASE ASGDB2;

O

CREATE DATABASE IF NOT EXISTS ASGDB2;

Agregar una columna

```
ALTER TABLE Clientes
ADD COLUMN email VARCHAR(150);
```

CAMBIAR EL NOMBRE DE UN CAMPO

```
ALTER TABLE Clientes
CHANGE COLUMN pais nacionalidad VARCHAR(100) NOT NULL;
```

ELIMINAR UNA COLUMNA

```
ALTER TABLE Clientes
DROP COLUMN edad;
```

MODIFICAR CAMPO

```
ALTER TABLE Clientes
MODIFY COLUMN nombre VARCHAR(200) NOT NULL;
```

ELIMINAR BASE DE DATOS

DROP DATABASE nombre_de_la_base;

ELIMINAR TABLA

DROP TABLE nombre_de_la_tabla;

Creación, modificación y eliminación de vistas

Las **vistas** son **consultas predefinidas** que presentan una parte específica de los datos almacenados en una o varias tablas. Son útiles para simplificar consultas complejas

Para crear una vista, utilizamos la sentencia `CREATE VIEW`. Definimos la consulta que deseamos como la base de la vista. Por ejemplo:

```
CREATE VIEW MiVista AS
SELECT nombre, edad FROM Clientes WHERE pais = 'España';
```

Modificación de vistas

Para modificar una vista existente utilizamos la sentencia ALTER VIEW. Por ejemplo:

```
ALTER VIEW MiVista AS
SELECT nombre, edad, ciudad FROM Clientes WHERE pais = 'España';
```

Ahora la vista incluye también la ciudad de los clientes.

Eliminación de vistas

Para eliminar una vista utilizamos la sentencia DROP VIEW. Por ejemplo:

DROP VIEW MiVista;

Elimina la vista «MiVista» de la base de datos.

Lenguaje de manipulación de datos (DML)

Operaciones DML sobre vistas

El **lenguaje de manipulación de datos (DML)** es un subconjunto de SQL *(structured query language)* utilizado para gestionar y manipular datos en una base de datos relacional.

Principales comandos DML

- **SELECT**: Consulta o recupera datos de una o varias tablas. Permite filtrar y ordenar la información.
- **INSERT**: Añade nuevas filas o registros a una tabla existente.
- **UPDATE**: Modifica los valores de los registros que ya existen en una tabla.
- **DELETE**: Borra filas o registros específicos de una tabla. *(Se suele usar con la cláusula WHERE para no eliminar toda la tabla)*.

Insertar 1 Registro

Insertar un solo registro:

- INSERT INTO Clientes (nombre, nacionalidad, edad, fecha_alta, email)
- VALUES ('Ana Pérez', ‘España', 28, '2024-05-10', 'ana.perez@email.com');

Insertar Varios registros

```
INSERT INTO Clientes (nombre, pais, edad, fecha_alta, email)
VALUES
('Carlos Ruiz', 'Chile', 35, '2023-11-15', 'carlos.ruiz@email.com'),
('Lucía Gómez', 'Argentina', 40, '2022-01-20', 'lucia.gomez@email.com'),
('Pedro Torres', 'España', 25, '2024-03-05', 'pedro.torres@email.com');
```

Modificar 1 Registro

```
UPDATE Clientes
SET email = 'ana.perez@empresa.com'
WHERE id = 1;
```

Modificar algunos registros

```
UPDATE Clientes
SET nacionalidad = ‘español'
WHERE nacionalidad = ‘España';
```

Aumentar la edad de los clientes en 1

```
UPDATE Clientes
SET edad = edad + 1;
```

Ver los datos

```
SELECT * FROM MiVista;
```

Eliminar datos

```
DELETE FROM Clientes WHERE id = 3;
```

Vaciar tabla más rápido (reinicia AUTO_INCREMENT)

TRUNCATE TABLE Clientes;

DCL *(data control language)*

reación de usuarios

En un SGBD, la gestión de usuarios es fundamental para garantizar la seguridad y el acceso controlado a la base de datos.

Creación de usuarios nuevos: para crear un nuevo usuario, utilizamos comandos DDL *(data definition language)* en SQL. Por ejemplo, podemos ejecutar: CREATE USER nombre_usuario IDENTIFIED BY 'contraseña'; Esto crea un usuario con el nombre especificado y la contraseña correspondiente.

Principales comandos DCL

- **GRANT**: Concede permisos específicos a un usuario o rol para que pueda realizar acciones en la base de datos (como leer, insertar o modificar tablas).
- **REVOKE**: Retira o elimina los permisos que se habían concedido previamente a un usuario o rol.

Ver los usuarios

```
SELECT user, host FROM mysql.user;
```

- CREATE USER 'usuario1'@'localhost' IDENTIFIED BY 'contraseña123’;

- CREATE USER 'usuario1'@'%' IDENTIFIED BY 'contraseña123';

Modificación de usuarios

Cambio de contraseña:

Si un usuario necesita cambiar su contraseña, podemos usar:

ALTER USER nombre_usuario IDENTIFIED BY 'nueva_contraseña’;

Cambio de nombre

```
RENAME USER 'usuario1'@'localhost' TO 'usuarioNuevo'@'localhost';
```

Tipos de permisos:

- SELECT: permite leer datos de una tabla o vista.
- INSERT: permite agregar nuevos registros a una tabla.
- UPDATE: permite modificar datos existentes en una tabla.
- DELETE: permite eliminar registros de una tabla.
- ALTER: permite modificar la estructura de una tabla (por ejemplo, agregar o eliminar columnas).
- GRANT: permite otorgar permisos a otros usuarios.

Asignación de privilegios: después de crear un usuario, asignamos privilegios específicos según las necesidades. Los privilegios pueden incluir permisos para acceder a tablas, vistas o ejecutar ciertas operaciones

GRANT ALL PRIVILEGES ON ASGDB3.CLIENTES2 TO ‘CESAR'@'localhost';

permitir solo **lectura (SELECT)**:

```
GRANT SELECT ON ASGDB2.* TO 'usuario1'@'localhost’;
```

Permitir **insertar y actualizar**:

```
GRANT INSERT, UPDATE ON ASGDB2.* TO 'usuario1'@'localhost';
```

Aplicar cambios

FLUSH PRIVILEGES;

```
SHOW GRANTS FOR 'usuario1'@'localhost’;
```

Modificación de privilegios

Para ajustar los privilegios de un usuario existente, utilizamos comandos como GRANT o REVOKE.

GRANT SELECT ON tabla_ejemplo TO nombre_usuario;

- **REVOCAR (quitar) PERMISOS**
- REVOKE ALL PRIVILEGES ON ASGDB2.* FROM 'usuario1'@'localhost';

Creación y eliminación de roles

- Los administradores pueden crear roles específicos según las necesidades de la organización.

CREATE ROLE Administradores;

rmisos a roles

```
GRANT SELECT, INSERT, UPDATE ON Clientes TO Administradores;

REVOKE INSERT, UPDATE ON ASGDB2.Clientes FROM
'administradores'@'localhost’;

SHOW GRANTS FOR 'rol’;
```

y retirada de roles a usuarios

Asignación de roles a usuarios:

- Los usuarios se agregan a roles para heredar sus permisos.

Por ejemplo, asignar al usuario «Ana» el rol «Administradores»: ALTER USER Ana ADD TO ROLE Administradores

Listar los usuarios

```
SELECT user, host FROM mysql.user;
```

Listar los roles

```
SELECT * FROM information_schema.enabled_roles;
```

CONCEPTO

Python y la conexion

1. Nuestro def de bd.py tiene la función conectar() que abre la comunicación con el servidor
2. Usando cursor() crea el objeto que enviará las sentencias.
3. execute() envía una sentencia SQL.
4. fetchone() o fetchall() recuperan resultados.
5. Cerrar cursor y conexión libera los recursos.

Leer solo dos columnas

Muestra el nombre y el precio de todos los productos, por id.

Antes de ver la solución, intenta explicar qué necesitas.
Pista: SELECT elige las columnas que queremos ver.

Leer solo dos columnas · solución

```
SELECT nombre, precio
FROM productos
ORDER BY id;
```

SELECT elige las columnas que queremos ver.
FROM indica la tabla de origen.
ORDER BY id fija un orden reproducible.

Resultado: Lápiz 1.50, Cuaderno 3.00 y Goma 0.80.

Filtrar productos baratos

Consulta los productos cuyo precio sea menor que 2 euros.

Antes de ver la solución, intenta explicar qué necesitas.
Pista: Primero elegimos nombre y precio.

Filtrar productos baratos · solución

```
SELECT nombre, precio
FROM productos
WHERE precio < 2
ORDER BY id;
```

Primero elegimos nombre y precio.
WHERE conserva las filas que cumplen la condición.
El número 2 se escribe sin comillas.

Resultado: Lápiz 1.50 y Goma 0.80. Cuaderno queda fuera.

Abrir una conexión

Ejecuta este programa. Comprueba que la conexión se abre y se cierra.

Antes de ver la solución, intenta explicar qué necesitas.
Pista: Importamos la función del archivo db.py incluido.

Abrir una conexión · solución

```
from db import conectar

with conectar() as conexion:
    print(conexion.is_connected())
print("Conexión cerrada")
```

Importamos la función del archivo db.py incluido.
with abre la conexión y la cierra al salir del bloque.
is_connected() devuelve True si se ha conectado.

Resultado: True, seguido de Conexión cerrada. Si falla, revisa la preparación.

Leer desde Python

Muestra las tres filas en Python usando execute y fetchall.

Antes de ver la solución, intenta explicar qué necesitas.
Pista: cn es la conexión. cur es el cursor que envía SQL.

Leer desde Python · solución

```
from db import conectar

with conectar() as cn:
    with cn.cursor() as cur:
        cur.execute("SELECT nombre, stock FROM productos ORDER BY id")
        filas = cur.fetchall()
        for fila in filas:
            print(fila)
```

cn es la conexión. cur es el cursor que envía SQL.
execute envía la consulta, pero no imprime las filas.
fetchall obtiene todas las filas y for las muestra una a una.

Resultado: Tuplas: (Lápiz, 10), (Cuaderno, 5) y (Goma, 20), con comillas en los textos.

Buscar por identificador

Busca el nombre del producto 2. Cambia después el id a 99.

Antes de ver la solución, intenta explicar qué necesitas.
Pista: %s representa un valor que enviamos por separado.

Buscar por identificador · solución

```
from db import conectar

with conectar() as cn:
    with cn.cursor() as cur:
        sql = "SELECT nombre FROM productos WHERE id = %s"
        cur.execute(sql, (2,))
        fila = cur.fetchone()
        print(fila)
```

%s representa un valor que enviamos por separado.
(2,) es una tupla de un elemento: necesita la coma.
fetchone devuelve una fila o None cuando no hay coincidencia.

Resultado: Con 2: ('Cuaderno',). Con 99: None. El id es único.

CONCEPTO

Si algo falla, revisa una cosa cada vez

ModuleNotFoundError: instala el conector en el Python correcto.
Conexión rechazada: comprueba servicio, host y puerto.
Access denied: revisa usuario, contraseña y host autorizado.
Unknown database: ejecuta la preparación.
Unread result found: consume las filas antes de otra consulta.

CONCEPTO

Cinco minutos para comprobar lo aprendido

1. ¿Qué diferencia hay entre fila y columna?
2. ¿Para qué sirve el cursor?
3. ¿Qué devuelve fetchone si no encuentra un id?
4. ¿Por qué (2,) lleva una coma?

Respuestas: registro/propiedad, enviar SQL, None, tupla de un elemento.

CONCEPTO

Banco de refuerzo opcional

Ejercicios E07–E12 para alumnado que termine antes, repaso o próxima sesión.
No se añaden a los 90 minutos previstos.
Cada enunciado va seguido de su solución explicada.

Ordenar por precio

Ordena los productos de menor a mayor precio.

Antes de ver la solución, intenta explicar qué necesitas.
Pista: ORDER BY precio usa orden ascendente por defecto.

Ordenar por precio · solución

```
SELECT nombre, precio FROM productos
ORDER BY precio, id;
```

ORDER BY precio usa orden ascendente por defecto.
Añadimos id para resolver posibles empates.

Resultado: Goma, Lápiz, Cuaderno.

Contar registros

Averigua cuántos productos hay.

Antes de ver la solución, intenta explicar qué necesitas.
Pista: COUNT(*) cuenta filas.

Contar registros · solución

```
SELECT COUNT(*) FROM productos;
```

COUNT(*) cuenta filas.
No cuenta columnas ni suma las existencias.

Resultado: 3 productos.

Buscar por texto

Muestra los nombres que empiezan por C.

Antes de ver la solución, intenta explicar qué necesitas.
Pista: LIKE compara un patrón.

Buscar por texto · solución

```
SELECT nombre FROM productos
WHERE nombre LIKE 'C%';
```

LIKE compara un patrón.
% representa cero o más caracteres.

Resultado: Cuaderno.

Usar un diccionario

Lee el producto 1 y muestra solo su nombre.

Antes de ver la solución, intenta explicar qué necesitas.
Pista: dictionary=True devuelve filas con claves de texto.

Usar un diccionario · solución

```
from db import conectar
with conectar() as cn:
    with cn.cursor(dictionary=True) as cur:
        cur.execute("SELECT nombre FROM productos WHERE id = %s", (1,))
        fila = cur.fetchone()
        print(fila["nombre"])
```

dictionary=True devuelve filas con claves de texto.
Accedemos a nombre mediante fila["nombre"].

Resultado: Lápiz.

Relacionar tablas

Muestra cada producto con el nombre de su categoría.

Antes de ver la solución, intenta explicar qué necesitas.
Pista: p y c son nombres cortos para las tablas.

Relacionar tablas · solución

```
SELECT p.nombre, c.nombre AS categoria
FROM productos p
JOIN categorias c ON p.categoria_id = c.id
ORDER BY p.id;
```

p y c son nombres cortos para las tablas.
ON empareja la clave foránea con la primaria.

Resultado: Lápiz, Cuaderno y Goma pertenecen a Papelería.

Crear una tabla sencilla

Crea una tabla notas con id y texto obligatorio.

Antes de ver la solución, intenta explicar qué necesitas.
Pista: CREATE TABLE define la estructura: es DDL.

Crear una tabla sencilla · solución

```
CREATE TABLE IF NOT EXISTS notas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    texto VARCHAR(100) NOT NULL
) ENGINE=InnoDB;
```

CREATE TABLE define la estructura: es DDL.
AUTO_INCREMENT genera ids. NOT NULL exige un valor.

Resultado: La tabla se crea vacía. Repetirlo no la duplica.